cw 1 weak

Machine Learning for Computer Vision Coursework One

C. David M. Hug
(CID 01303626)

Samuel Riedel
(CID 01288992)

Abstract

The intention of this report is to explore capabilities and limitations of the randomized decision forest algorithm. Particularly, it is concerned with its implementation, the development of a fundamental understanding of its principles and the influence of chosen parameters and split functions, splitting methods respectively.

1. Introduction

The random forest algorithm is based on an ensemble of individual decision trees. Each of which implements an arbitrary classification algorithm that takes a predefined feature vector as an input and generates an according class label. The random forest algorithm was initially researched by Tin Kam Ho. Subsequent refined and augmented research was conducted by Leo Breiman and Adele Cutler. Specifically, the initial motivation [1] as to reduce the over-fitting of single decision trees and/or any other appropriate classification method by exploiting averaging of a multitude of individual decision trees that had been trained on data-subsets. Due to the fact that there are still loads of possible improvements to be made, the general area of randomized decision forests is still a widely popular research topic in machine learning and computer vision.

2. Training Decision Forests

2.1. Bagging Algorithm

First of all, some implementation of bagging algorithm needs to be employed to split the original dataset $\mathcal{S}_0$ into different subsets $\mathcal{S}_0^i$. There are several different methods and parameters that can be used to do so. In particular, subsets may be created with or without replacement and using a variable percentage of the original number of training points within each subset. Each decision tree is trained on a different subset $\mathcal{S}_0^i$, hence, the number of subsets that need to be created directly depends on the number of desired trees. The employed bagging method that has been used throughout the coursework, is presented in figure 1. Bagging was done [3] sing replacement and 63.2% of the dataset $\mathcal{S}_0$.

[4]

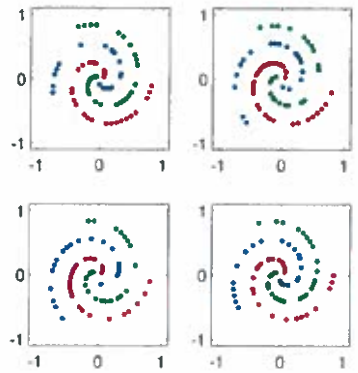


Figure 1: Four different example sub-sets that have been created by bagging (with replacement) using a sampling fraction of $1 - (1/e) \approx 63.2\%$.

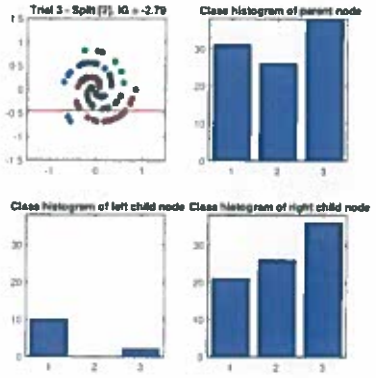


Figure 2: Illustration of a specific split function. As can be seen, it has been managed to completely eliminate class two in the left child node.

2.2. Employed Split Functions

Due to the given code structure for this subtask, [2] e split functions does split the two-dimensional subset either in x_1 or x_2 direction. The split is chosen in such way that the information gain for a specific split is maximized in an empirical manner (1). Specifically, the algorithm tries out a

Summary of Comments on mlcv_cw1_weak.pdf

Page: 1

Number: 1 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:34:30

Number: 2 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:35:53

Number: 3 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:35:33

Number: 4 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:39:06

good!

finite number of threshold values τ and takes the one that maximizes the information gain $\mathcal{I}(\mathcal{S}_0, \Theta(x_{1\ldots n}, \tau))$ (where n is equal to two in this case).

$$\mathcal{I}(\mathcal{S}_0, \Theta(x_{1\ldots n}, \tau)) = H(\mathcal{S}_0) - \sum_{i \in \text{L,R}} \frac{|\mathcal{S}_0^i|}{|\mathcal{S}_0|} H(\mathcal{S}_0^i) \quad (1)$$

One example of a possible split function is shown in figure 2. In this particular instance, the information gain is around -2.79 and $\tau \approx \{\emptyset, -0.5\} = \{x_1^a, x_2^a\} \in \mathbb{R}^2$. It is worth mentioning that the split function can be chosen in an arbitrary way. Exempli gratia, the split function could also be a straight skew line of the form $\Theta(x_1, x_2, \tau) = \text{sgn}(\tau_1 x_1 + \tau_2 - x_2)$ or even a spiral like function (in polar coordinates) $\Theta(r, \varphi, \tau)$. However, this would be equivalent to data snooping and should only be used with great care.

2.3. Class distribution of leaf nodes

Next, a short discussion, that is concerned with the outputs of the individual trees, is carried out. The stopping criterion to create the actual leaf nodes was either limited by the chosen depth of the tree d_T or the minimal number of remaining data points m_p that are necessary to give rise to yet another split into a left and/or right child node. For instance, if we start with 13 data points and the split function splits the set into nodes with 9 (left child node) and 4 (right child node) remaining data points, the right node is considered to be a leaf of the tree. In our implementation, $m_p = 6$ was chosen.

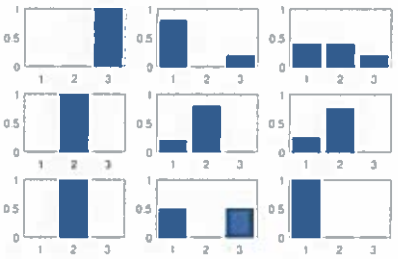


Figure 3: Possible outcome for some leave nodes of an arbitrary tree. The bars in the illustration show the class distribution in a leaf node, that is, they show the probability that the feature vector/data point that served as input to a tree and arrived at a particular node belongs to a certain class.

Due to the fact that we have a decision tree forest, there exist a multitude of different trees with different parameters (e.g. depth, split values et cetera). Hence, the class distribution for other trees in the forest may look unalike the one that is presented because splits within nodes are carried out in a different way altogether.

3. Evaluating Decision Forests on Test Data

Using the previously trained randomized decision forest, one is now able to estimate or predict the classification of any valid new data point. This is done by passing the new feature vector (e.g. coordinates of the point in the Cartesian plane) to the decision forest and letting the decision forest decide on the class that the new point should be categorized in. Of course, due to several limitations (e.g. size of training data, tree depth, split function types), this is only an estimate of the true class label and the point may be misclassified by the forest. In order to visualize how well the previously trained decision forest performs, an evaluation/-classification of new points that lie on a (relatively) dense grid may be used. Due to the fact that this toy example uses three classes that may be represented by red, blue and green, one is able to transform the class distribution of every new test point to a single RGB pixel within an image. An illustrative example of a possible resulting picture is shown in figure 4 (where the number of trees is 20, the tree depth is 8 and the split parameter number is 2). Figure 5 shows the class distribution of four points generated by eight trees and the average of all trees.

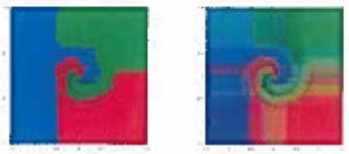


Figure 4: Class classification/distribution on a dense grid that has been transformed into a RGB picture for better visual interpretation. The presented picture has an accuracy of 100.00% on the training data and its computational cost was quite low (e.g. < 1 [min] CPU time).

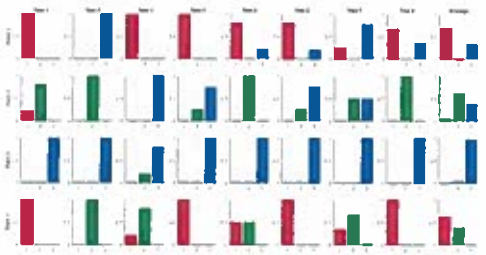


Figure 5: Class distributions of the leaf nodes which the data points arrive at and the averaged class distribution. Data points: Point 1: $(-0.5, -0.7)$, Point 2: $(0.4, 0.3)$, Point 3: $(-0.7, 0.4)$, Point 4: $(0.5, -0.5)$

3.1. Number of Trees

Individual trees tend to either not capture the complexity of the data set that lies at hand or tend to be over-fitted to

Page: 2

Number: 1 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:36:22

Number: 2 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:38:52
this part is good!

Number: 3 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:36:52

Number: 4 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:36:55

Number: 5 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 20:32:13

Number: 6 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:38:07
Do we earn higher information gain when using linear ones than axis-aligned?
Are the histograms tended to be peaky? What does that mean? in terms of entropy/information gain and classification in the end (see those of leaf nodes).
Do we compare different options of stopping?

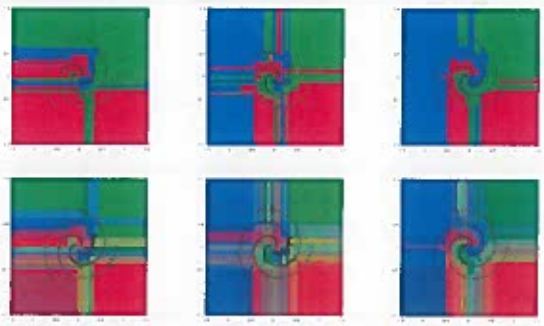


Figure 6: *Dependency between classification accuracy* ***(top)****, class distribution* ***(bottom)*** *respectively, and the involved number of trees (where $d_T = 12$ and $s_T = 12$ is fixed).* ***Left:*** *$n_T = 2$,* ***Middle:*** *$n_T = 4$,* ***Right:*** *$n_T = 6$.*

the given data. One way to improve the performance of the decision forest is to employ an increasing amount of trees. The idea behind this procedure is that individual tree may be neither accurate nor well balanced. However, in an ensemble of trees, the average prediction is quite accurate nonetheless. The empirical proof that this prediction holds can be seen in figure 6. The computational cost increases linearly with increasing number of tree n_T.

3.2. Tree Depth

As would have been expected, the trees depth does positively correspond to the accuracy of the predicted classification of new test points that arrive (see Figure 7). However, it is worth mentioning that computational effort does get big quite fast as it grows exponentially with the parameter d_T and exposes a complexity of $\mathcal{O}(2^{d_T})$. It is worth pointing out as well that, even though classification seems to always get better if only the presented illustrations are considered, individual trees tend to over-fit with increasing depth as well. This effect may be reduced/averaged out by using a large ensemble of trees.

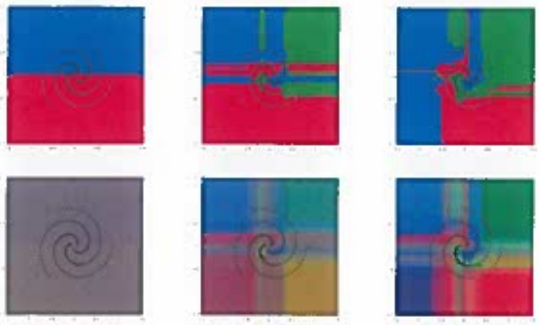


Figure 7: *Dependency between classification accuracy* ***(top)****, class distribution* ***(bottom)*** *respectively, and depth of the individual trees (where $n_T = 20$ and $s_T = 12$ is fixed).* ***Left:*** *$d_T = 2$,* ***Middle:*** *$d_T = 4$,* ***Right:*** *$d_T = 8$.*

3.3. Number of Splits/Randomness Parameter

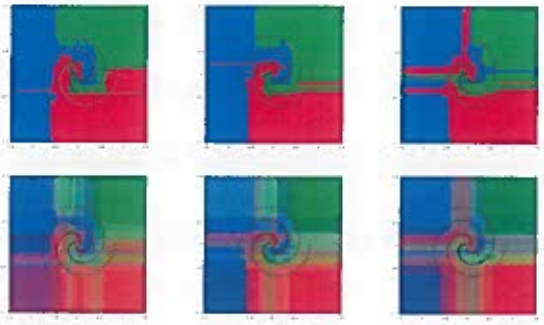


Figure 8: *Dependency between classification accuracy* ***(top)****, class distribution* ***(bottom)*** *respectively, and number of split attempts (where $n_T = 10$ and $d_T = 6$ is fixed).* ***Left:*** *$s_T = 2$,* ***Middle:*** *$s_T = 4$,* ***Right:*** *$s_T = 8$.*

A short investigation regarding the effect of the number of splits that are tried has been made as well and is presented in 8. It can be seen from the graphical evidence that the number of splits increases the accuracy up to a certain value. However, when the number of splits is increased to even higher values, the resulting classification tends to over-fit and results in small class junks that are disconnected from the principal class cluster. The parameter s_T leads to additional complexity that scales with $\mathcal{O}(s_T)$. Furthermore, different split functions have been implemented and tested. The considered split functions are either axis-aligned, two pixel test (TPT) or both. Results of illustrative nature are presented in figure 9.

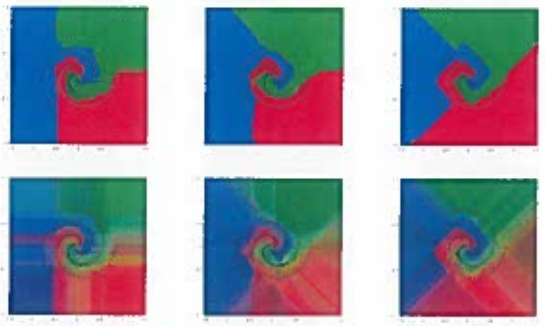


Figure 9: *Classification* ***(top)*** *and class distribution* ***(bottom)*** *for different implementations of a split function (where $n_T = 20$, $d_T = 8$ and $s_T = 2$ is fixed).* ***Left:*** *Axis-aligned,* ***Middle:*** *Axis-aligned or diagonal/TPT,* ***Right:*** *Diagonal/TPT.*

4. Experiments with the Caltech101 Dataset

In the following part of the report, experiments based on the Caltech101 dataset have been conducted. Particularly, different implementations of image categorization have been

Page: 3

Number: 1 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 20:32:41
very good

Number: 2 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 20:32:37

Number: 3 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 20:32:54
good discussions

Number: 4 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 20:33:20
this part is excellent

Number: 5 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 20:34:15
what if we try to observe their combinatorial impacts?

made. Firstly, the well known K-means algorithm was employed to generate feature data of different images classes and, secondly, a randomized forest codebook was used to extract crucial feature data of given image classes.

4.1. K-Means Codebook

Vocabulary Size The number of SIFT descriptors that are involved in the subsequent computations have been capped at 100'000 different features by default. Each SIFT descriptor is a $[8 \times 4 \times 4]$ 3D histogram and hence the resulting (stacked) descriptor vector that is generated by each SIFT can be seen as a single $[128 \times 1]$ descriptor vector. Due to the fact that computations get slowed down if descriptor vectors of such length are used, the first thing that was used during the implemented algorithm, that is responsible for the generation of the k-means codebook, is a principle component analysis (PCA). Although each entry of the SIFT histogram has a particular meaning, we just consider the stacked descriptor vector as an ordinary feature vector. Intuition, and several empirically obtained measurements, indicate that many components of the original SIFT descriptor/feature vectors are not helpful in achieving successful classification of new data. This is due to the fact that the variance of some SIFT descriptor vector components may be relatively small. Hence, these low variance dimensions might be at least very hard to split, if not entirely useless for further significant extrapolation. During some experimentation, it has been discovered that merely the 32 most important PCA transformed SIFT descriptor vector components do suffices to result in almost the same test accuracy as the original 128 descriptor vector whilst reducing the computational effort significantly (Figure 10). The mathematical expression to determine the new feature/descriptor vector with 32 principal components is stated in equation (2).

$$X^{32} = X^{128}\, \mathcal{PCA}(X^{128}_{(:,1\ldots32)}) \tag{2}$$

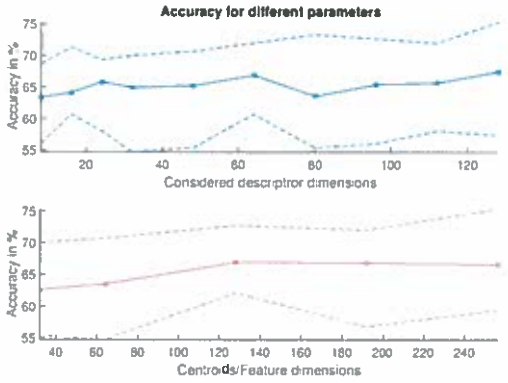


Figure 10: Example confusion matrix for $n_T = 250$, $d_T = 8$, $s_T = 3$.

Bag-of-Words Histograms The next step in the implemented algorithm is to find a certain number of centroids to spacially group the aforementioned 32 dimensional (reduced) descriptor vector into clusters that then serve as definite features dimensions of some picture. In order to find the necessary clusters/feature groups, the Matlab function `kmeans(...)` was used to find 128 clusters. Once again, by using some empirical testing, the aforementioned number of clusters was found to perform quite well without having too many features/histogram bins that do not generate useful additional information to increase performance on the test data. Of course, the selected number of bins within the histogram can be chosen arbitrarily, however, a trade-off between accuracy and computational cost has to be made. For simplicity, it has been assumed that 128 bins deliver a good balance between accuracy and computational cost throughout the rest of this coursework (Figure 10).

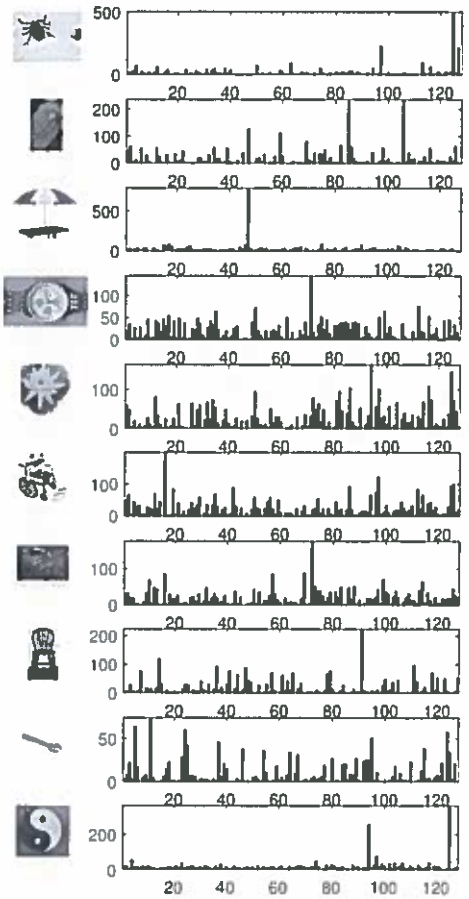


Figure 11: Example feature histogram, 32 considered PCA descriptor dimensions and 128 histogram dimensions.

Page: 4

Number: 1 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 18:56:11

Number: 2 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 18:56:23

Number: 3 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 18:56:39
a very good point

Number: 4 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 18:56:31

Number: 5 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 18:54:27

Number: 6 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 18:57:47
do we have any time measurement? for computational cost..

Number: 7 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 18:55:59
do we vary this parameter value?

Number: 8 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 18:55:34

Number: 9 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 18:54:56
very nice trial

Vector Quantization Process To create the histogram/feature vector for the images, the nearest centroid was determined for each descriptor vector of a particular image. Each histogram represents the number of times that the descriptor vectors have been assigned to one of the previously determined centroids. For illustrative purposes, several histograms of arbitrary training pictures for all different classes are presented in figure 11.

4.2. Randomized Forest Classifier

Using the feature vectors created as described above, a random decision tree forest was trained to categorize the images. To find optimal, or at least well performing, tree parameters (n_T,d_T and s_T) a large number of randomized decision forests have been grown and their average accuracy for a selection of parameters is shown in figure 12. The upper and lower dotted lines represent the maximum and minimum measured accuracy, whilst the solid line denotes the average case. Note, the same set of training and testing data was used throughout this parameter sweep. Comparing the accuracy for different number of trees, it can be seen, that the curve begins to saturate at around 150 trees. Saturation effects can also be observed with respect to the maximal tree depth. Particularly, if the trees depth is increased beyond 8 only slightly improvements can be achieved. When looking at the accuracy for different numbers of attempted (axis aligned) splits, one can see that there seems to be an optimum at around three. It is worth pointing out that this number is fairly low and implies that the trees tend to overfit when optimizing the split parameters to much. A more randomly chosen split might be advantageous.

Different Split Functions Both types of split functions (axis-aligned and PT) have been tested as well, however, the measured in-/decrease is not significant. This result applies as well for the RF codebook that will be discussed shortly.

Confusion Matrix and Accuracy The confusion matrix of the best measurement that has been taken for some well performing parameters is shown in figure 13. The recognition accuracy tends to be quite good in average, if the parameters are chosen in a reasonable manner. It is worth mentioning again that all the calculations have been carried out using the reduced descriptor vector with merely 32 dimensions and 128 centroids/feature dimensions for the histogram (see figure 10). The main classes that get mixed up are mostly classes three, four and seven which correspond to the umbrella, the watch and the wild cat class. Target classes two and eight (Trilobite and Windsor chair) do perform exceptionally well as all pictures get classified correctly.

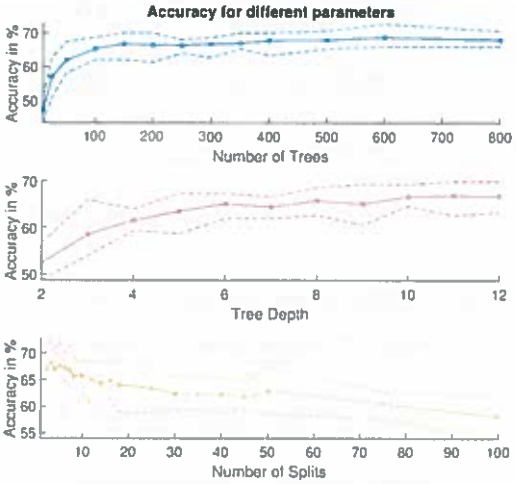


Figure 12: *Accuracy for different tree parameters* $n_T = 250$, $d_T = 8$, $s_T = 3$

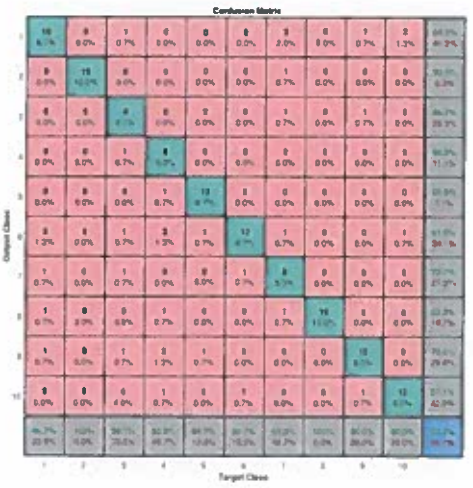


Figure 13: *Example confusion matrix for* $n_T = 250$, $d_T = 8$, $s_T = 3$, *32 considered PCA descriptor dimensions*

Time-efficiency A priori, one needs to realize that there is a huge set of possible parameters and implementations that can be chosen, which may lead to quite divergent results with respect to accuracy and computational efficiency. In the following, a brief discussion is only conducted for the parameters that have been used to generate the confusion matrix shown in figure 13. Besides the stated parameters that have already been written down, the k-means algorithm has been set up to carry out two replications whilst determining the disjunct clusters. This has been done in an attempt to find a better local minimum. In this particular instance, the entire image training and categorization finished within one minutes time. 44% of the time was

Page: 5

Number: 1 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 18:58:58
very good

Number: 2 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 18:58:25
good!

Number: 3 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 18:58:47

Number: 4 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 18:59:07

Number: 5 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 18:59:28
excellent statistical analysis on the performance

Number: 6 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 18:59:53

Number: 7 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 19:00:30

Number: 8 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:00:46
some results using different types of split functions?

Number: 9 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 19:01:38

sed to extract the PHOW/SIFT descriptors, around 37% of the time was needed to cluster the selected descriptors and 15% of the time was used to grow the randomized forest. The remaining time was used for loading the images, testing the trees, vector quantization and doing PCA transformations. Using PCA the computation time of k-means algorithm could be largely reduced in particular (e.g. from around 52 [s] to around 15 [s] using only one replication). It needs to be pointed out that similar accuracies can often be achieved using much less trees, much lower tree depth and fewer splits, whilst tremendously reducing the computational effort (see figure 12).

Vocabulary size and accuracy The impact of the vocabulary size on the classification accuracy has already been discussed in the previous and redundant justification will be omitted here. The effects are illustrated in figure 10.

4.3. Randomized Forest Codebook

Vocabulary Generation The main difference between the use of a random forest codebook and a k-means codebook is the fact that a random forest is used for clustering. Specifically this is done by, growing multiple trees using the descriptor vectors as training data. Once again, PCA is employed on the selection of descriptors. To create the vocabulary, all the leaf nodes in the forest are labeled uniquely (consecutively numbered). Therefore, the feature dimension is equal to the number of leaf nodes. To assign a histogram/feature vector to a specific image, we input its descriptor vectors to the tree and according to the implemented split functions, each descriptor vector gets assigned to a leaf node. A histogram is then calculated containing all the leaf node labels of all descriptor vectors of a picture.

Classifier Now, that the histograms/feature vector has been assigned to each image, the adapted algorithm follows the previous implementation very closely. Specifically, a random forest was trained using the newly defined histograms. The training and testing of the random forest is exactly equivalent to the previous implementation.

Accuracy Discussion The achieved accuracy using the random forest codebook is somewhere between 55% and 66%. The confusion matrix of the resulting run with the highest accuracy is shown in figure 14. Compared to the accuracy using the k-means codebook, the random forest codebook is roughly 10% less accurate and there is more fluctuation in the individual results. A possible explanation for this decrease in accuracy could be due to the fact that, k-means splits the space into proper disjunct classes, whilst the randomized forest approach leads to multi-labeling of some spacial regions. This circumstance may be one cause for the decrease in performance. Additionally, while the k-means algorithms tries to cluster points that are in proximity to each other, the randomized forest approach may chose splits that separate data points that would have been consolidated before.

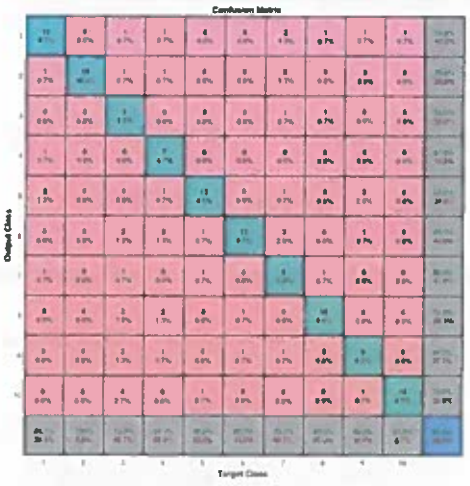


Figure 14: Example confusion matrix for $n_T = 50$, $d_T = 8$, $s_T = 3$, 32 considered PCA descriptor dimensions. Note: Codebook forest and classifier forest are constructed using the same parameters

Efficiency Discussion In terms of computation time, the randomized decision forest codebook takes quite some time to generate. During the fastest attempt that avoids taking a severe hit on accuracy, 90 [s] of 95 [s] in total, were used to generate the codebook. 30% of the codebook generation time was used to extract the PHOW/SIFT descriptors, around 22% of the time was needed to grow the random forest used for feature generation and 19% of the time was invested in calculating the histograms of each image. It was a bit astounding that the loading of images took longer than usually for this measurement as well. Growing the final randomized decision trees and classifying the test data is almost negligible compared to the codebook generation. In comparison with the k-means codebook creation, the randomized forest attempt is significantly slower.

Vector Quantization Complexity Due to the fact, that each leaf node is seen as a separate feature dimension, the number of bins in the pictures histogram is growing extremely fast with increasing three depth. The complexity order is $\mathcal{O}(2^{d_T})$. Hence, the analysis of each picture and therefore, the generation of its histogram takes much more time, as a huge number of parameters is involved. One might expect that the binary tree based classification would be faster than the nearest neighbor search, however Matlabs implementation of the `knnsearch(...)` is extremely fast and outperforms even the fast test tree function.

Page: 6

Number: 1 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:04:19
what are pros and cons of bigger vocabulary size? in terms of efficiency? Does a bigger one give us a better accuracy always? (overfitting, noise etc) Do we have some such observations?
do we measure the time for vector quantisation? for different values of K?

Number: 2 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:01:52
a good analysis on the training time

Number: 3 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 19:01:38

Number: 4 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:06:35
what do we observe when changing multiple (more than 1) parameters at once? e.g. 3D plot (z: performance, x: parameter 1, y: parameter 2)
do we measure both training time and test time for some different RF parameter values?

Number: 5 Author: Tae-Kyun Subject: Highlight Date: 25/05/2017 10:13:02

Number: 6 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:13:47
strange... worth checking the concept and respective implementation

Number: 7 Author: Tae-Kyun Subject: Highlight Date: 25/05/2017 10:13:13

Number: 8 Author: Tae-Kyun Subject: Highlight Date: 25/05/2017 10:12:13

Number: 9 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:12:26
interesting, a good observation

Number: 10 Author: Tae-Kyun Subject: Highlight Date: 25/05/2017 10:12:37

Number: 11 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:12:43
a good trial

Number: 12 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:14:29
okay conclusions

Number: 13 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:15:12
do we try different parameters in both RF codebook and RF classifier? what are the best parameters in each? do we observe any difference, if yes, why?
do we obtain any improvements in terms of accuracy, why?
do we measure the vector quantization time for RF codebook, in comparison with k-means? what split function do we use? axis-aligned? or linear??

5. Appendix

```
%% Initialize and load data

% Simple Random Forest Toolbox for Matlab
% written by Mang Shao and Tae-Kyun Kim, June 20, 2014.
% updated by Tae-Kyun Kim, Feb 09, 2017

% This is a guideline script of simple-RF toolbox.
% The codes are made for educational purposes only.
% Some parts are inspired by Karpathy's RF Toolbox

% Under BSD Licence

% Initialisation
init;

%% Select dataset
[data_train, data_test] = getData('Toy_Spiral'); % {'Toy_Gaussian', 'Toy_Spiral', 'Toy_Circle', '
    Caltech'}

% Question 1 and 2
% Tests on the dense 2D grid data, and visualises the results

% Set the random forest parameters for instance
param.num = 20;          % Number of trees
param.depth = 8;         % trees depth
param.splitNum = 2;      % Number of split functions to try
param.split = 'IG';      % Currently support 'information gain' only

% Grow all trees
trees = growTrees(data_train,param);
%trees = growTreesTPT(data_train,param);

% Plots and images
mod_data_test = zeros(size(data_test,1),6);
for n=1:size(data_test,1)
    leaves = testTrees([data_test(n,:) 0],trees);
    %leaves = testTreesTPT([data_test(n,:) 0],trees);
    p_rf = trees(1).prob(leaves(leaves~=0),:);
    p_rf_sum = sum(p_rf)/length(trees);
    [~, idx] = max(p_rf_sum);
    mod_data_test(n,:) = [data_test(n,1:2), idx, p_rf_sum];
end

figure(1);
plot_toydata(data_train);
axis equal;
axis tight;

figure(2);
img = zeros(151,151,3);
img(:,:,1) = reshape(mod_data_test(:,4),[151, 151]);
img(:,:,2) = reshape(mod_data_test(:,5),[151, 151]);
img(:,:,3) = reshape(mod_data_test(:,6),[151, 151]);
imagesc([-1.5 1.5], [-1.5 1.5], img);
hold on;
plot_toydata(data_train);
set(gca,'ydir','normal');
axis equal;
axis tight;

figure(3);
gscatter(mod_data_test(:,1),mod_data_test(:,2),mod_data_test(:,3));
img(:,:,1) = reshape(mod_data_test(:,3)==1,[151, 151]);
```

```
63  img(:,:,2) = reshape(mod_data_test(:,3)==2,[151, 151]);
64  img(:,:,3) = reshape(mod_data_test(:,3)==3,[151, 151]);
65  imagesc([-1.5 1.5], [-1.5 1.5], img);
66  hold on;
67  plot_toydata(data_train);
68  set(gca,'ydir','normal');
69  axis equal;
70  axis tight;
71
72  %% Question 3: Experiment with Caltech101 dataset for image categorisation
73
74  % Select dataset
75  % we do bag-of-words technique to convert images to vectors (histogram of codewords)
76  % Set 'showImg' in getData.m to 0 to stop displaying training and testing images and their feature
        vectors
77  [data_train, data_test] = getData('Caltech');
78  close all;
79
80  % Set the random forest parameters for instance
81  param.num = 250;          % Number of trees
82  param.depth = 8;          % trees depth
83  param.splitNum = 5;       % Number of split functions to try
84  param.split = 'IG';       % Currently support 'information gain' only
85
86  % Train Random Forest
87  trees = growTrees(data_train,param);
88  %trees = growTreesTPT(data_train,param);
89
90  % Evaluate/Test Random Forest
91  mod_data_test = zeros(size(data_test,1),size(data_test,2) + max(data_train(:,end)) + 1);
92  for n=1:size(data_test,1)
93      leaves = testTrees(data_test(n,1:end-1),trees);
94      %leaves = testTreesTPT(data_test(n,1:end-1),trees);
95      p_rf = trees(1).prob(leaves(leaves~=0),:);
96      p_rf_sum = sum(p_rf)/length(trees);
97      [~, idx] = max(p_rf_sum);
98      mod_data_test(n,:) = [data_test(n,1:end), idx, p_rf_sum];
99  end
100
101 clearvars n leaves p_rf p_rf_sum idx
102
103 % Shows accuracy and confusion matrix
104 % ind = size(data_test,2);
105 % conf = zeros(max(data_train(:,end)),max(data_train(:,end)));
106 % for n = 1:size(data_test,1)
107 %     conf(mod_data_test(n,ind),mod_data_test(n,ind + 1)) = conf(mod_data_test(n,ind),mod_data_test
        (n,ind + 1)) + 1;
108 % end
109 % accuracy = trace(conf)/sum(conf(:));
110 %
111 % imagesc(conf);
112 % title(sprintf('Confusion matrix (%.2f %% accuracy)', 100 * accuracy));
113
114 ind = size(data_test,2);
115 targets = zeros(max(data_train(:,end)), size(data_test,1));
116 outputs = zeros(max(data_train(:,end)), size(data_test,1));
117 for n = 1:size(data_test,1)
118     targets(mod_data_test(n,ind),n) = 1;
119     outputs(mod_data_test(n,ind + 1),n) = 1;
120 end
121 plotconfusion(targets,outputs);
122
123 clearvars accuracy conf ind n targets outputs
124
125 %% Random forest codebook for Caltech101 image categorisation
126 [data_train, data_test] = getData('CaltechRF');
127 close all;
```

```
128
129 % Set the random forest parameters for instance
130 param.num = 50;          % Number of trees
131 param.depth = 8;         % trees depth
132 param.splitNum = 3;      % Number of split functions to try
133 param.split = 'IG';      % Currently support 'information gain' only
134
135 % % Train Random Forest
136 trees = growTrees(data_train,param);
137 %trees = growTreesTPT(data_train,param);
138
139 % Evaluate/Test Random Forest
140 mod_data_test = zeros(size(data_test,1),size(data_test,2) + max(data_train(:,end)) + 1);
141 for n=1:size(data_test,1)
142     leaves = testTrees(data_test(n,1:end-1),trees);
143     %leaves = testTreesTPT(data_test(n,1:end-1),trees);
144     p_rf = trees(1).prob(leaves(leaves~=0),:);
145     p_rf_sum = sum(p_rf)/length(trees);
146     [-, idx] = max(p_rf_sum);
147     mod_data_test(n,:) = [data_test(n,1:end), idx, p_rf_sum];
148 end
149
150 clearvars n leaves p_rf p_rf_sum idx
151
152 ind = size(data_test,2);
153 targets = zeros(max(data_train(:,end)), size(data_test,1));
154 outputs = zeros(max(data_train(:,end)), size(data_test,1));
155 for n = 1:size(data_test,1)
156     targets(mod_data_test(n,ind),n) = 1;
157     outputs(mod_data_test(n,ind + 1),n) = 1;
158 end
159 plotconfusion(targets,outputs);
160
161 clearvars accuracy conf ind n targets outputs
```

Listing 1: *Matlab code for* `main`

```
1  function [ data_train, data_query ] = getData( MODE )
2  % Generate training and testing data
3
4  % Data Options:
5  %   1. Toy_Gaussian
6  %   2. Toy_Spiral
7  %   3. Toy_Circle
8  %   4. Caltech 101
9
10 showImg = 0; % Show training & testing images and their image feature vector (histogram
       representation)
11
12 PHOW_Sizes = [4 8 10]; % Multi-resolution, these values determine the scale of each layer.
13 PHOW_Step = 8; % The lower the denser. Select from {2,4,8,16}
14
15 switch MODE
16     case 'Toy_Gaussian' % Gaussian distributed 2D points
17         %rand('state', 0);
18         %randn('state', 0);
19         N= 150;
20         D= 2;
21
22         cov1 = randi(4);
23         cov2 = randi(4);
24         cov3 = randi(4);
25
26         X1 = mgd(N, D, [randi(4)-1 randi(4)-1], [cov1 0;0 cov1]);
27         X2 = mgd(N, D, [randi(4)-1 randi(4)-1], [cov2 0;0 cov2]);
```

```
        X3 = mgd(N, D, [randi(4)-1 randi(4)-1], [cov3 0;0 cov3]);

        X= real([X1; X2; X3]);
        X= bsxfun(@rdivide, bsxfun(@minus, X, mean(X)), var(X));
        Y= [ones(N, 1); ones(N, 1)*2; ones(N, 1)*3];

        data_train = [X Y];

    case 'Toy_Spiral' % Spiral (from Karpathy's matlab toolbox)

        N= 50;
        t = linspace(0.5, 2*pi, N);
        x = t.*cos(t);
        y = t.*sin(t);

        t = linspace(0.5, 2*pi, N);
        x2 = t.*cos(t+2);
        y2 = t.*sin(t+2);

        t = linspace(0.5, 2*pi, N);
        x3 = t.*cos(t+4);
        y3 = t.*sin(t+4);

        X= [[x' y']; [x2' y2']; [x3' y3']];
        X= bsxfun(@rdivide, bsxfun(@minus, X, mean(X)), var(X));
        Y= [ones(N, 1); ones(N, 1)*2; ones(N, 1)*3];

        data_train = [X Y];

    case 'Toy_Circle' % Circle

        N= 50;
        t = linspace(0, 2*pi, N);
        r = 0.4
        x = r*cos(t);
        y = r*sin(t);

        r = 0.8
        t = linspace(0, 2*pi, N);
        x2 = r*cos(t);
        y2 = r*sin(t);

        r = 1.2;
        t = linspace(0, 2*pi, N);
        x3 = r*cos(t);
        y3 = r*sin(t);

        X= [[x' y']; [x2' y2']; [x3' y3']];
        Y= [ones(N, 1); ones(N, 1)*2; ones(N, 1)*3];

        data_train = [X Y];

    case {'Caltech','CaltechRF'} % Caltech dataset
        close all;
        imgSel = [15 15]; % randomly select 15 images each class without replacement. (For both
            training & testing)
        folderName = './Caltech_101/101_ObjectCategories';
        classList = dir(folderName);
        classList = {classList(3:end).name} % 10 classes

        disp('Loading training images...')
        % Load Images -> Description (Dense SIFT)
        cnt = 1;
        if showImg
            figure('Units','normalized','Position',[.05 .1 .4 .9]);
            suptitle('Training image samples');
        end
```

```
94          for c = 1:length(classList)
95              subFolderName = fullfile(folderName,classList{c});
96              imgList = dir(fullfile(subFolderName,'*.jpg'));
97              imgIdx{c} = randperm(length(imgList));
98              imgIdx_tr = imgIdx{c}(1:imgSel(1));
99              imgIdx_te = imgIdx{c}(imgSel(1)+1:sum(imgSel));
100
101             for i = 1:length(imgIdx_tr)
102                 I = imread(fullfile(subFolderName,imgList(imgIdx_tr(i)).name));
103
104                 % Visualise
105                 if i < 6 & showImg
106                     subaxis(length(classList),5,cnt,'SpacingVert',0,'MR',0);
107                     imshow(I);
108                     cnt = cnt+1;
109                     drawnow;
110                 end
111
112                 if size(I,3) == 3
113                     I = rgb2gray(I); % PHOW work on gray scale image
114                 end
115
116                 % For details of image description, see http://www.vlfeat.org/matlab/vl_phow.html
117                 [~, desc_tr{c,i}] = vl_phow(single(I),'Sizes',PHOW_Sizes,'Step',PHOW_Step); %
                        extracts PHOW features (multi-scaled Dense SIFT)
118             end
119         end
120
121         switch MODE
122             case 'Caltech'
123                 disp('Building visual codebook...')
124                 % Build visual vocabulary (codebook) for 'Bag-of-Words method'
125                 desc_sel = single(vl_colsubset(cat(2,desc_tr{:}), 1e5)); % Randomly select 100k
                        SIFT descriptors for clustering
126
127 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% USER CODE %%%%%%%%%%%%%%%%%%%%
```

```
            case 'CaltechRF'

                disp('Building visual codebook...')
                % Build visual vocabulary (codebook) for 'Bag-of-Words method'
                descPerClass = round(1e5 / length(classList));
                desc_sel = NaN(descPerClass * length(classList),128 + 1);
                for i = 0:length(classList)-1
                    desc_sel((descPerClass*i+1):(descPerClass*(i+1)),:) = [transpose(single(
                        vl_colsubset(cat(2,desc_tr{i+1,:}), descPerClass))), (i+1) * ones(
                        descPerClass,1)]; % Randomly select 10k SIFT descriptors per class
                end

                % PCA on data selection
                consDim = 32;
                pcaTransDesc = pca(desc_sel(:,1:end-1));
                pcaTransDesc = pcaTransDesc(:,1:consDim);
                desc_sel = [desc_sel(:,1:end-1) * pcaTransDesc, desc_sel(:,end)];

                % Delete zero rows
                desc_sel = desc_sel(any(desc_sel(:,1:end-1),2), :);

                % Set the random forest parameters for instance
                param.num = 50;          % Number of trees
                param.depth = 8;          % Trees depth
                param.splitNum = 3;      % Number of split functions to try
                param.split = 'IG';       % Currently support 'information gain' only

                % Grow all trees
                trees = growTrees(desc_sel,param);
                %trees = growTreesTPT(desc_sel,param);
                numNodes = trees(end).leaf(end).label;

                disp('Encoding Images...')
                desc_tr = transpose(desc_tr);
                data_train = zeros(length(classList) * length(imgIdx_tr),numNodes + 1);
                for i = 1:size(desc_tr,1) * size(desc_tr,2)

                    % Test trees
                    temp = testTrees_fast(single(transpose(desc_tr{i})) * pcaTransDesc,trees);
                    %temp = testTrees_fastTPT(single(transpose(desc_tr{i})) * pcaTransDesc,trees);

                    % Generate Output
                    data_train(i,:) = [histcounts(temp,1:numNodes+1), ceil(i/imgSel(1))];

                end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% END USER CODE %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

        end

        % Clear unused varibles to save memory
        clearvars desc_tr desc_sel
end

switch MODE
    case {'Caltech','CaltechRF'}
        if showImg
        figure('Units','normalized','Position',[.05 .1 .4 .9]);
        suptitle('Testing image samples');
        end
        disp('Processing testing images...');
        cnt = 1;
        % Load Images -> Description (Dense SIFT)
        for c = 1:length(classList)
            subFolderName = fullfile(folderName,classList{c});
            imgList = dir(fullfile(subFolderName,'*.jpg'));
            imgIdx_te = imgIdx{c}(imgSel(1)+1:sum(imgSel));
```

```
223
224             for i = 1:length(imgIdx_te)
225                 I = imread(fullfile(subFolderName,imgList(imgIdx_te(i)).name));
226
227                 % Visualise
228                 if i < 6 & showImg
229                     subaxis(length(classList),5,cnt,'SpacingVert',0,'MR',0);
230                     imshow(I);
231                     cnt = cnt+1;
232                     drawnow;
233                 end
234
235                 if size(I,3) == 3
236                     I = rgb2gray(I);
237                 end
238                 [-, desc_te{c,i}] = vl_phow(single(I),'Sizes',PHOW_Sizes,'Step',PHOW_Step);
239
240             end
241         end
242         if showImg
243         figure('Units','normalized','Position',[.5 .1 .4 .9]);
244         suptitle('Testing image representations: 256-D histograms');
245         end
246
247 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% USER CODE %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
248
249         switch MODE
250             case 'Caltech'
251                 % Vector quantisation: Generates the features for the pictures
252                 desc_te = transpose(desc_te);
253                 data_query = zeros(length(classList) * length(imgIdx_te),numBins + 1);
254                 for i = 1:(size(desc_te,1) * size(desc_te,2))
255
256                     % Search nearest neighbour
257                     temp = knnsearch(centroids,transpose(single(desc_te(i))) * pcaTransDesc);
258
259                     % Generate output
260                     data_query(i,:) = [histcounts(temp,1:numBins+1), ceil(i/imgSel(2))];
261
262                 end
263
264             case 'CaltechRF'
265
266                 desc_te = transpose(desc_te);
267                 data_query = zeros(length(classList) * length(imgIdx_te),numNodes + 1);
268                 for i = 1:size(desc_te,1) * size(desc_te,2)
269
270                     % Test trees
271                     temp = testTrees_fast(single(transpose(desc_te(i))) * pcaTransDesc,trees);
272                     %temp = testTrees_fastTPT(single(transpose(desc_te(i))) * pcaTransDesc,trees);
273
274                     % Generate Output
275                     data_query(i,:) = [histcounts(temp,1:numNodes+1), ceil(i/imgSel(2))];
276
277                 end
278
279                 % PCA on trees
280                 data_query = [data_query(:,1:end-1), data_query(:,end)]; %* pcaTransTree
281
282         end
283
284 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% END USER CODE %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
285
286     otherwise % Dense point for 2D toy data
287         xrange = [-1.5 1.5];
288         yrange = [-1.5 1.5];
289         inc = 0.02;
```

```
290         [x, y] = meshgrid(xrange(1):inc:xrange(2), yrange(1):inc:yrange(2));
291         data_query = [x(:) y(:) zeros(length(x)^2,1)];
292 end
293 end
```

***Listing 2:** Matlab code for* `get_data`

```
1  function tree = growtreesTPT(data,param)
2
3  %        Base                Each node stores:
4  %         1                    trees.idx       - data (index only) which split into this node
5  %        / \                   trees.t         - threshold of split function
6  %       2   3                  trees.dim       - feature dimension of split function
7  %      / \ / \                 trees.prob      - class distribution of this node
8  %     4  5 6  7                trees.leaf_idx  - leaf node index (empty if it is not a leaf node)
9
10 disp('Training Random Forest...');
11
12 [N,D] = size(data);
13 frac = 1 - 1/exp(1); % Bootstrap sampling fraction: 1 - 1/e (63.2%)
14
15 cnt_total = 1;
16 [labels,~] = unique(data(:,end));
17 for T = 1:param.num
18
19     % Bootstraping aggregating
20     idx = randsample(N,ceil(N*frac),1);
21     % A new training set for each tree is generated by random sampling from dataset WITH
           replacement.
22     prior = histc(data(idx,end),labels)/length(idx);
23
24     % Initialise base node
25     tree(T).node(1) = struct('idx',idx,'t',nan,'dim',-1,'prob',[]);
26
27     % Split Nodes
28     for n = 1:2^(param.depth-1)-1
29         [tree(T).node(n),tree(T).node(n*2),tree(T).node(n*2+1)] = splitNodeTPT(data,tree(T).node(n)
               ,param);
30     end
31
32     % Leaf Nodes
33     cnt = 1;
34     for n = 1:2^param.depth-1
35         if ~isempty(tree(T).node(n).idx)
36             % Percentage of observations of each class label
37             tree(T).node(n).prob = histc(data(tree(T).node(n).idx,end),labels)/length(tree(T).node(
                   n).idx);
38
39             if ~tree(T).node(n).dim % if this is a leaf node
40                 tree(T).node(n).leaf_idx = cnt;
41                 tree(T).leaf(cnt).label = cnt_total;
42                 prob = reshape(histc(data(tree(T).node(n).idx,end),labels),[],1);
43
44                 tree(T).leaf(cnt).prob = prob; %.*prior; % Multiply by the prior probability of
                       bootstrapped sub-training-set
45                 tree(T).leaf(cnt).prob = tree(T).leaf(cnt).prob./sum(tree(T).leaf(cnt).prob); %
                       Normalisation
46
47                 if strcmp(param.split,'Var')
48                     tree(1).cc(cnt_total,:) = mean(data(tree(T).node(n).idx,1:end-1),1); % For RF
                           clustering, unfinished
49                 else
50                     tree(1).prob(cnt_total,:) = tree(T).node(n).prob';
51                 end
52
```

```
                cnt = cnt+1;
                cnt_total = cnt_total + 1;
            end
        end
    end
end
end
```

Listing 3: Matlab code for `growTreesTPT`

```
function [node,nodeL,nodeR] = splitNodeTPT(data,node,param)
% Split node

visualise = 0;

% Initilise child nodes
iter = param.splitNum;
nodeL = struct('idx',[],'t',nan,'dim',0,'prob',[]);
nodeR = struct('idx',[],'t',nan,'dim',0,'prob',[]);

if length(node.idx) <= 5 % make this node a leaf if has less than 5 data points
    node.t = nan;
    node.dim = 0;
    return;
end

idx = node.idx;
data = data(idx,:);
[N,D] = size(data);
ig_best = -inf; % Initialise best information gain
idx_best = [];
for n = 1:iter

    % Split function - Modify here and try other types of split function
    dim = randi(D-1); % Pick one random dimension
    dim2 = randi(D-1);
    dir = [-1, 1];
    dir = dir(randperm(2));
    dir = dir(1);
%     while dim == dim2
%         dim2 = randi(D-1);
%     end
    d_min = single(sqrt(2) * min(data(:,dim))) + eps; % Find the data range of this dimension
    d_max = single(sqrt(2) * max(data(:,dim))) - eps;
    t = d_min + rand*((d_max-d_min)); % Pick a random value within the range as threshold
    idx_ = (data(:,dim) < t + dir * data(:,dim2));

    ig = getIG(data,idx_); % Calculate information gain

    if visualise
        visualise_splitfunc(idx_,data,dim,t,ig,n);
        pause();
    end

    [node, ig_best, idx_best] = updateIG(node,ig_best,ig,t,idx_,[dim,dim2,dir],idx_best);

end

nodeL.idx = idx(idx_best);
nodeR.idx = idx(~idx_best);

if visualise
    visualise_splitfunc(idx_best,data,dim,t,ig_best,0)
    fprintf('Information gain = %f. \n',ig_best);
    pause();
```

```
end

end

function ig = getIG(data,idx) % Information Gain - the 'purity' of data labels in both child nodes
    after split. The higher the purer.
L = data(idx);
R = data(~idx);
H = getE(data);
HL = getE(L);
HR = getE(R);
ig = H - sum(idx)/length(idx)*HL - sum(~idx)/length(idx)*HR;
end

function H = getE(X) % Entropy
cdist= histc(X(:,1:end), unique(X(:,end))) + 1;
cdist= cdist/sum(cdist);
cdist= cdist .* log(cdist);
H = -sum(cdist);
end

function [node, ig_best, idx_best] = updateIG(node,ig_best,ig,t,idx,dim,idx_best) % Update
    information gain
if ig > ig_best
    ig_best = ig;
    node.t = t;
    node.dim = dim;
    idx_best = idx;
else
    idx_best = idx_best;
end
end
```

***Listing 4:** Matlab code for* `splitNodeTPT`

```
function label = testTrees_fastTPT(data,tree)
% Faster version - pass all data at same time
%disp('Testing Random Forest...');

cnt = 1;
for T = 1:length(tree)
    idx{1} = 1:size(data,1);
    for n = 1:length(tree(T).node);
        if ~tree(T).node(n).dim
            leaf_idx = tree(T).node(n).leaf_idx;
            if ~isempty(tree(T).leaf(leaf_idx))
                label(idx{n}',T) = tree(T).leaf(leaf_idx).label;
            end
            continue;
        end
        idx_left = data(idx{n},tree(T).node(n).dim(1)) < tree(T).node(n).t + tree(T).node(n).dim(3)
            .* data(idx{n},tree(T).node(n).dim(2));
        idx{n*2} = idx{n}(idx_left');
        idx{n*2+1} = idx{n}(~idx_left');
    end
end

end
```

***Listing 5:** Matlab code for* `testTrees_fastTPT`

```
function label = testTreesTPT(data,tree)
% Slow version - pass data point one-by-one

```

```
cc = [];
for T = 1:length(tree)
    for m = 1:size(data,1);
        idx = 1;

        while tree(T).node(idx).dim
            t = tree(T).node(idx).t;
            dim = tree(T).node(idx).dim;
            % Decision
            if data(m,dim(1)) < t + dim(3) * data(m,dim(2)) % Pass data to left node
                idx = idx*2;
            else
                idx = idx*2+1; % and to right
            end

        end
        leaf_idx = tree(T).node(idx).leaf_idx;

        if ~isempty(tree(T).leaf(leaf_idx))
            p(m,:,T) = tree(T).leaf(leaf_idx).prob;
            label(m,T) = tree(T).leaf(leaf_idx).label;

%             if isfield(tree(T).leaf(leaf_idx),'cc') % for clustering forest
%                 cc(m,:,T) = tree(T).leaf(leaf_idx).cc;
%             end
        end
    end
end

end
```

***Listing 6:** Matlab code for* `testTreesTPT`